

if an integer and it expected a double, the conversion would be no problem--converting an integer into a double doesn't lose any information. However, if I passed it a double and it was expecting an integer, the conversion from double to integer WOULD lose information, so the compiler would complain. The compiler would complain unless I used an explicit cast operator to force the argument into the right type.

### 3.2 Duration of Identifiers

The duration of an identifier is the lifetime of the identifier. Identifiers declared locally in a method are called automatic. Automatic variables exist only while the block they are declared in executes. Static variables exist from the time the class that defines them is loaded into memory until the program terminates.

### 3.3 Scope Rules

The scope for an identifier is the portion of the program in which the identifier can be referenced. The scopes for an identifier are:

- class
- block
- method

### 3.4 Method Overloading

Method overloading allows a method name to be re-used. To overload a method--or to create another version of a method that already exists--the argument lists for the methods must differ in:

- number of arguments
- type of arguments
- order of arguments
- The return type of the method is NOT considered.

### 3.5 Event Delegation Model

Unlike many other languages--such as Visual Basic--when the Java programmer wishes to program responses to an event--such as the enter key being pressed, the event must be entirely programmed.

Consider this analogy to the event model:

- Steve Jobs decides he wants a bodyguard.
- Somebody punches Steve--that's a problem.
- Bill--the bodyguard--notices Steve was punched.